

Claude Bug Bounty

Como extrair o máximo do plugin — do recon ao report — usando Caído como proxy. Fluxo de trabalho, comandos, regras e exemplos reais.

Preparado para Daniel · caça de bugs em HackerOne · Bugcrowd · Intigriti · Immunefi

Em uma frase: este projeto transforma o Claude Code num assistente de bug bounty com memória, uma metodologia de 5 fases, ~30 comandos de barra e 13 skills especializadas. Você conduz; ele faz recon, sugere ataques, valida achados e escreve o report — sempre dentro do escopo que você definir.

1. O que é e como pensar sobre ele

Não é um scanner. É uma **metodologia guiada**. A filosofia central do projeto:

“Hunting não é *achar um bug* — é *provar um cenário de ataque*. Pense como um atacante com um objetivo específico, não como um scanner procurando padrões.”

As três peças que você vai usar o tempo todo:

Peça	O que é	Como você aciona
Comandos (/...)	Ações diretas: recon, hunt, validate, report...	Digita <code>/recon target.com</code> no prompt
Skills	Bases de conhecimento que carrego sob demanda (21 classes de bug, payloads, templates de report)	Eu carrego sozinho, ou você força com <code>/bug-bounty</code>
Memória	Aprende com cada alvo: padrões que funcionaram, leads em aberto	<code>/remember</code> , e o "lead board" automático

Regra de ouro do projeto: automação só para *recon* (subdomínios, hosts vivos, URLs). Teste manual é o que acha bug único — scanner automático acha duplicata. Você pilota o teste; eu acelero o trabalho repetitivo.

2. Setup — o que você já tem pronto

Item	Status
Proxy Caído + MCP (66 tools de leitura/replay do histórico)	✅ Conectado
Servidor MCP correto (<code>c0tton-fluff/caído-mcp-server</code> , binário Go)	✅ Instalado em <code>~/go/bin</code>
Push do seu fork via HTTPS (conta <code>danielnogueira84</code>)	✅ Configurado

Único hábito diário: abra o **Caido antes** do Claude Code. O MCP só conecta com o Caido aberto na porta 8080. Se aparecer `Connection closed`, 99% das vezes é o Caido fechado. Teste rápido:

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080
# 200 = ok    |    000 = Caido fechado
```

3. A regra que vem antes de tudo: ESCOPO

Uma requisição fora de escopo = risco de ban. Um report fora de escopo = fechado na hora. O projeto tem um verificador **determinístico** (`tools/scope_checker.py` — checagem por código, não "achismo" de IA).

Importante sobre o Caido: puxar do histórico *não* filtra por escopo automaticamente — o histórico traz todo o tráfego que você capturou. O escopo é aplicado quando eu vou *agir*. Por isso, no começo de cada alvo:

```
# Puxa todos os assets in-scope de H1/Bugcrowd/Intigriti/etc.
/scope-aggregate nome-do-programa

# Confere um asset específico antes de tocar
/scope app.target.com
```

4. O fluxo de trabalho — 5 fases não-lineares

[1. RECON] → [2. MAPEAR] → [3. ACHAR] → [4. PROVAR] → [5. REPORTAR]
▲──────────────────|──────────────────|
Travou numa fase? Volte para a anterior. Achou API nova na fase 3? Volte à 2.

Fase 0 — antes de tocar em qualquer ferramenta, responda 3 perguntas:

1. **Define:** "Hoje eu ataco [feature/domínio] para conseguir [Confidencialidade / Integridade / Disponibilidade / ATO / RCE]"
2. **Select:** escolha 1–2 classes de bug (IDOR, race condition, SSRF...). Não saia vagando.
3. **Identity:** anônimo ou autenticado? Se caça IDOR/BOLA/priv-esc, carregue a sessão uma vez no início.

Fase 1 — Recon (maximizar superfície)

Objetivo: achar o que os outros não acharam. Comando principal: `/recon target.com` — roda subdomínios, hosts vivos (httpx), portas, URLs (gau/katana), análise de JS e fuzzing de diretórios, tudo para `recon/<target>/`.

Achou...	Próximo passo
Subdomínios vivos com stack conhecida (WordPress, Jira)	Checar CVEs e defaults na hora → <code>/scan-cves host</code>
Recursos de nuvem (S3, Firebase)	<code>/cloud-recon --keyword nome</code>
403 ou 200 + página de bloqueio	<code>/bypass-403 <url></code> (detecta soft-block)
Nada depois de 5 min num host	Pula (regra dos 5 minutos)

Fase 2 — Mapear (entender o app como o dev)

- Mapeie todos os endpoints (sitemap do Caido + análise de JS)
- Identifique o modelo de auth (cookie, JWT, OAuth, SAML?)
- Ache os fluxos críticos: pagamento, cadastro, reset de senha, exportação de dados
- Baixe e analise os bundles JS: rotas escondidas, segredos, lógica
- Anote comportamentos "estranhos" (anomalias de nomes, erros, timing)

Ferramentas úteis aqui: `/param-discover <url>` (params escondidos = ouro para IDOR/SSRF), `/secrets-hunt --js-bundle recon/<target>`, `/surface target.com` (ranking de superfície).

Fase 3 — Achar o bug (erro primeiro, cego depois)

Roteie pelo *tipo de input* que está testando:

O input é...	Ataque
Parâmetro de ID (<code>user_id</code> , <code>order_id</code>)	IDOR / BOLA
Busca / filtro / ordenação	SQLi, NoSQLi
URL / webhook / gerador de PDF	SSRF
Texto refletido na página	XSS (DOM ou refletido)
Upload de arquivo	SVG XSS, web shell, path traversal
Preço / quantidade / cupom	Lógica de negócio, race condition
Login / 2FA / reset de senha	Bypass de autenticação
Endpoint GraphQL	<code>/graphql-audit <url></code>

Ordem de detecção: Erro (`'` , `"` , `{{7*7}}`) → Tempo (`SLEEP(10)`) → OOB (callback DNS) → Booleano (`1=1` vs `1=0`).

Fase 4 — Provar e escalar (Low → Critical)

Achar o bug é metade. O que paga é o **impacto**. Exemplos de escalada do próprio projeto:

Achou	Escalada
XSS	Rouba cookie/token → sequestro de sessão → ATO. HttpOnly? Força troca de e-mail via XHR → ATO
IDOR que lê PII	Automatiza a coleta, mostra escala. Troca senha/e-mail? → ATO direto
SSRF	Só DNS? Não reporta . Chega em <code>169.254.169.254</code> ? → extrai chaves de nuvem → RCE
Open redirect	No fluxo OAuth? → roubo de token → ATO

Quando confirmar o bug A → **pare** e cace B e C por 20 min (o mesmo dev errou de novo). Use `/chain` .

Fase 5 — Validar e reportar (receber)

```
/validate # roda o 7-Question Gate. Qualquer falha = mata o achado.
/triage   # go/no-go rápido para casos borderline
/report   # gera o report no formato da plataforma, CVSS, PoC
/remember # grava o padrão na memória do alvo
```

O report ideal: título no formato *[Classe] em [Endpoint] permite [papel] fazer [impacto]*, primeira frase = o que o atacante CONSEGUE fazer, requests HTTP exatos, menos de 600 palavras, CVSS que bate com o impacto real.

5. Catálogo de comandos por fase

Fase	Comando	Para quê
Escopo	<code>/scope <asset></code>	Confere se um asset está em escopo
	<code>/scope-aggregate <prog></code>	Puxa todos os assets in-scope das plataformas
Recon	<code>/recon target.com</code>	Pipeline completo de recon
	<code>/surface target.com</code>	Superfície de ataque priorizada
	<code>/param-discover <url></code>	Parâmetros HTTP escondidos
	<code>/secrets-hunt ...</code>	Credenciais vazadas (JS, git, org GitHub)
	<code>/cloud-recon ...</code>	Buckets públicos + IP de origem atrás de CloudFlare
	<code>/takeover --recon <dir></code>	Candidatos a subdomain takeover
Achar / explorar	<code>/hunt target.com</code>	Dispara o pipeline de teste de vulns
	<code>/graphql-audit <url></code>	Auditoria completa de GraphQL
	<code>/bypass-403 <url></code>	38+ técnicas de bypass de 403/401
	<code>/scan-cves <host></code>	Varredura nucleí de CVEs high/critical
	<code>/intel target.com</code>	CVEs + reports divulgados do alvo
Chain / validar / reportar	<code>/chain</code>	Monta cadeia A→B→C para elevar severidade
	<code>/validate · /triage</code>	Mata achado fraco antes do report
	<code>/report</code>	Report pronto para submissão
Memória / sessão	<code>/pickup target.com</code>	Retoma um hunt anterior
	<code>/remember</code>	Grava achado/padrão na memória
Web3 / tokens	<code>/web3-audit <contrato.sol></code>	Auditoria de smart contract (10 classes)
	<code>/token-scan <contrato></code>	Scanner de rug pull / meme coin
Autônomo	<code>/autopilot target.com --normal</code>	Loop autônomo: recon→rank→hunt→validate→report

Todos os comandos são prefixados para não colidir com os nativos do Claude Code. `/resume` é reservado — use `/pickup` para retomar um hunt.

6. Exemplos de ponta a ponta

Exemplo A — alvo novo, do zero ao report

```
# 1. Escopo primeiro (sempre)
/scope-aggregate acme-corp      # lista assets in-scope
/scope app.acme.com            # confirma o que vou testar

# 2. Recon
/recon app.acme.com

# 3. Ranquear a superfície e escolher o alvo do dia
/surface app.acme.com
# Defino: "hoje ataco o fluxo de convites de time p/ conseguir IDOR"

# 4. (navego o app pelo Caído, capturando tráfego)
#   Peço a análise do histórico:
"analisa meu histórico do Caído no host app.acme.com e liste
 endpoints com IDs sequenciais candidatos a IDOR"

# 5. Encontro /api/v2/team/123/members -> testo o irmão:
#   /api/v2/team/123/export com meu token e ID de outro time
# 6. Confirmado -> escalo e valido
/validate
/report
```

Exemplo B — trabalhando com o histórico do Caído

Com o Caído aberto e o MCP conectado, em vez de colar requests você conversa com o tráfego que já capturou:

```
"liste os POSTs para *.acme.com no meu histórico do Caído que
mandam JSON com campos de preço ou quantidade"

"pegue o request GET /api/user/settings do histórico e me diga
quais parâmetros escondidos vale testar para mass assignment"
```

Leitura do histórico está na allowlist (não pede confirmação). Já o *envio* de requests para o alvo (`caído_send_request`) eu confirmo com você antes — de propósito, para você aprovar tráfego a alvo.

Exemplo C — transformando um achado fraco em pagável

```
# Achei um open redirect isolado (normalmente $0)
/chain
# Eu procuro um "conector": se o alvo usa OAuth, o open redirect
# no redirect_uri vira roubo de token -> ATO (Critical).
```

7. As regras que mais economizam tempo

#	Regra	Por quê
1	Leia o escopo completo antes do 1º request	1 request fora = ban em potencial
2	Nunca cace bug teórico	"Pode teoricamente..." não é bug. Machuca sua taxa de validade
3	Mate achado fraco rápido (7-Question Gate)	Cada minuto num achado fraco = um a menos num real
5	Regra dos 5 minutos	Superfície sem nada em 5 min → próxima
10	Regra do irmão (sibling)	Se <code>/user/123/orders</code> pede auth, teste <code>/export</code> , <code>/delete</code> , <code>/share</code> . Explica 30% dos IDORs pagos
11	Método A→B	Bug confirmado = sinal de que o dev errou em classe. Ache B em 20 min
12	Novo == não revisado	Features com menos de 30 dias têm a menor maturidade de segurança
13	Siga o dinheiro	Billing/créditos/refund/carteira = onde há mais atalho de dev
14	Rotação de 20 min	"Estou progredindo?" Não → rotacione endpoint/subdomínio/classe
17	Vazamento de credencial precisa de prova	Achar uma API key = Informational. Provar o que ela acessa = Medium/High

Anti-padrões (pare de fazer):

- **Pular de programa:** fique num alvo no mínimo 2 semanas / 30 horas.
- **Caçar só com ferramenta:** automação acha duplicata; manual acha bug único.
- **Toca de coelho:** máximo 45 min por parâmetro. Travou? Durma sobre o assunto.
- **Sem objetivo:** "só dando uma olhada" = tempo perdido. Sempre *Define* primeiro.

8. Como me pedir as coisas (para render mais)

Em vez de...	Peça assim
"acha bugs nesse site"	"defini o alvo X, classe IDOR — liste endpoints do meu histórico do Caido com IDs sequenciais e proponha os requests irmãos a testar"
"esse achado é bom?"	" <code>/validate</code> " — eu rodo o 7-Question Gate e mato ou aprovo
"escreve o report"	" <code>/validate</code> primeiro; se passar, <code>/report</code> para HackerOne"
"analisa esse contrato"	" <code>/web3-audit Contrato.sol</code> " (aplica kill-signals antes de mergulhar)

Boas perguntas de alto sinal que eu respondo bem: "quais os 10 erros de trust-boundary mais prováveis nesse endpoint?", "que endpoints irmãos / métodos / papéis testar em seguida?", "se esse bug é real, que bug B e C ficam ao lado?", "qual o menor request que prova isso?"

Memória: eu já registrei seu setup (Caido em vez de Burp; push pessoal via HTTPS na conta [danielnogueira84](#)). Conforme você caça, uso [/remember](#) e o "lead board" para não perder nenhum lead entre sessões. Use [/pickup target.com](#) para retomar um alvo depois.

9. Bônus — Web3 e Tokens (blockchain)

Além do web tradicional (web2), o projeto cobre bug bounty em **blockchain / cripto** — útil sobretudo para programas da **Immunefi**, onde os pagamentos são altos. São dois domínios distintos.

9.1 Web3 — auditoria de smart contracts

Foco: achar bugs em contratos DeFi (Solidity/Rust) que permitam roubar fundos. Comando: `/web3-audit Contrato.sol`. A skill aplica primeiro os **kill-signals** (ex.: TVL < US\$500K → não vale o esforço) e traz template de PoC em Foundry. As 10 classes, em ordem de frequência real:

Classe	O que é
Accounting desync (28%)	A contabilidade interna do contrato sai de sincronia com o saldo real de tokens
Access control (19%)	Função crítica sem checagem de permissão
Incomplete path / off-by-one	Caminho de código não tratado; erro de 1 unidade em cálculos (22% dos Highs)
Oracle manipulation	Manipular preço (via flash loan) para drenar fundos
Reentrancy	Chamar de volta antes de atualizar o saldo — o clássico
ERC4626 attacks	Ataques de inflação de share em cofres (vaults)
Flash loan oracle	Empréstimo instantâneo para distorcer um oráculo de preço
Signature replay · Proxy/upgrade	Reuso de assinatura; falhas em contratos atualizáveis

9.2 Tokens / meme coins — detecção de rug pull

Foco: avaliar se um token é golpe (honeypot / rug pull) — serve para bounty e para *due diligence* antes de investir. Cobre EVM (Solidity) e Solana (SPL / Token-2022). Comandos: `/token-scan <contrato>` ou `/token-scan <contrato> --chain solana`, que rodam o `token_scanner.py` automático + auditoria manual de 8 classes.

Red flag	O golpe
Hidden mint	O dono pode imprimir tokens infinitos depois do lançamento
Honeypot	Você compra mas não consegue vender
Fee manipulation	O dono muda a taxa de venda para 100%
Fake renounce / authority retention	Finge ter "renunciado" o controle, mas mantém a autoridade (mint/freeze)
LP lock bypass / LP drain	Retira a liquidez travada, deixando o token sem valor
Bonding curve exploit · sandwich/MEV	Explora a curva de preço (pump.fun/Raydium) ou amplifica ataques de sandwich

Quando usar cada um: web3 responde "esse contrato DeFi tem um bug que dá para roubar fundos?"; token responde "essa moeda é um golpe?". Se o seu foco é web app (o setup com Caído deste guia), esta seção é opcional — mas está aqui caso um programa da Immunefi ou uma análise de token apareça.

10. Checklist de bolso

Início de sessão	Durante	Fim de sessão
☐ Abrir Caido ☐ <code>/scope</code> do alvo ☐ Define/Select/Identity ☐ <code>/pickup</code> se já visitado	☐ Regra do irmão em todo endpoint ☐ Relógio de 20 min ☐ A→B ao confirmar um bug ☐ <code>/bypass-403</code> se travar em WAF	☐ Salvar projeto do Caido ☐ <code>/validate</code> + <code>/report</code> ☐ <code>/remember</code> os achados ☐ Anotar tentativas que falharam

Guia gerado a partir da metodologia real do plugin (skills/bb-methodology, rules/hunting.md e CLAUDE.md do projeto). Uso autorizado em programas de bug bounty com safe-harbor. Nunca teste assets fora de escopo.